

Flutter 项目中 Git 代码同步与分支管理

在 Flutter 项目中使用 Git 保持办公室与家里的代码一致，核心是通过 **远程仓库同步代码**，结合分支管理和规范的提交流程来实现。以下是具体步骤和最佳实践：

一、准备工作：初始化仓库与关联远程

1. 本地初始化 Git 仓库（如果项目未使用 Git）

在项目根目录打开终端，执行：

```
git init # 初始化本地仓库
git add . # 将所有文件加入暂存区（注意先配置 .gitignore）
git commit -m "初始化项目：第一个提交" # 提交到本地仓库
```

关键：先创建 `.gitignore` 文件，忽略不需要提交的文件（如 Flutter 缓存、IDE 配置等），可参考 [GitHub 官方 Flutter .gitignore](#)。

2. 创建远程仓库

在 GitHub、GitLab、Gitee 等平台新建一个远程仓库（如 `flutter_project`），获取仓库地址（HTTPS 或 SSH，推荐 SSH 免密）。

3. 关联本地仓库到远程

终端执行：

```
git remote add origin 远程仓库地址 # 关联远程仓库（origin 是远程仓库的默认别名）
git push -u origin main # 首次推送本地 main 分支到远程，并关联跟踪
```

二、日常同步流程（办公室 → 家里 / 家里 → 办公室）

核心原则：**每次开始工作前拉取最新代码，结束工作后推送本地修改。**

场景 1：从办公室到家里

1. 办公室提交代码

下班前，确保本地修改已提交并推送到远程：

```
git status # 检查修改的文件
git add 文件名 # 或 git add . 暂存所有修改
git commit -m "描述修改: 例如 修复登录页bug / 添加首页轮播组件" # 提交到本地
git push origin main # 推送到远程 main 分支
```

2. 家里拉取代码

到家后，在本地项目目录执行：

```
git pull origin main # 拉取远程最新代码到本地（自动合并，若有冲突需解决）
```

场景 2：从家里到办公室

与上述流程相反：

1. 家里修改后，commit 并 push 到远程。
2. 到办公室后，git pull origin main 获取最新代码。

三、进阶：多分支管理（避免多人协作或功能开发冲突）

如果需要开发新功能或修复 bug，建议创建分支，避免直接在 main 分支修改，流程如下：

1. 创建并切换到新分支（如开发一个 "个人中心" 功能）：

```
git checkout main # 确保基于最新的 main 分支
git pull origin main # 拉取最新代码
git checkout -b feature/user-center # 创建并切换到功能分支
```

2. 在分支上开发

完成后提交并推送到远程分支：

```
git add .
git commit -m "完成个人中心页面布局"
git push origin feature/user-center # 推送分支到远程（首次推送需指定分支名）
```

3. 切换设备后继续开发该分支

换电脑（如家里）后，拉取远程分支并切换：

```
git pull origin feature/user-center # 拉取远程分支到本地
git checkout feature/user-center # 切换到该分支继续开发
```

4. 功能完成后合并到 main 分支

功能测试通过后，合并到主分支：

```
git checkout main # 切回主分支
git pull origin main # 确保主分支最新
git merge feature/user-center # 合并功能分支到主分支
git push origin main # 推送合并后的主分支到远程
```

四、解决冲突（关键操作）

当两地修改了同一文件的同一部分，`git pull` 或 `git merge` 时会出现冲突，解决步骤：

1. 终端会提示 "Automatic merge failed", 此时打开冲突文件，会看到类似标记：

```
<<<<<<< HEAD
办公室修改的内容
=====
家里修改的内容
>>>>>> feature/user-center
```

2. 手动编辑文件，保留正确内容，删除冲突标记（<<<<<<<、=====、>>>>>>>）。

3. 解决后提交：

```
git add 冲突文件 # 标记为已解决
git commit -m "解决合并冲突：保留xxx内容"
git push origin main # 推送解决后的代码
```

五、推荐工具

- **命令行**：最基础，适合熟悉 Git 操作的场景。

-

图形化工具：如 GitKraken、SourceTree，或 IDE 集成的 Git 工具（Android Studio/VS Code 均有），可视化操作更直观，适合处理冲突。

通过以上流程，可确保办公室和家里的代码始终保持同步，核心是 **"先拉后改，改完即推"**，配合分支管理可减少冲突，提高效率。

（注：文档部分内容可能由 AI 生成）